BANKON Subname Registrar — Architecture, Tokenomics & Winning Strategy

An ENS-native architecture for the agent economy, built on NameWrapper, x402, and KeeperHub.

Bottom line up front

A `bankon.eth` subname registrar built on ENS NameWrapper is the single best-fit submission for the ETHGlobal OpenAgents ENS track, because the prize sponsor's verbatim copy — *"ENS is how you give them a name, a reputation, and a place to be found"* — describes exactly the Soul/Mind/Hands stack PYTHAI already operates. ENS Labs published a November 2025 blog post explicitly endorsing **ENS + ERC-8004 + x402** as the canonical agent-identity stack, and BANKON is that reference implementation.

The recommended build is a **Foundry-based L1 NameWrapper registrar** with **medium-tier ENS-aligned annual rent** (3-char $320 / 4-char $80 / 5+ char $5), an **x402 multi-chain payment gateway** routing USDC on Base and PYTHAI ASA on Algorand back to L1 via an EIP-712 voucher relayer, **bundled ERC-8004 identity mint** so every BANKON subname is a first-class ERC-8004 agent, and **KeeperHub as the onchain execution layer** for autonomous renewal, PYTHAI buyback-and-make, BONAFIDE-driven slashing, and agent-to-agent workflow invocation.

This document specifies the architecture, the tokenomics design space, complete Solidity scaffolding, the KeeperHub integration, and a 4-minute demo flow that satisfies the ENS, KeeperHub, and 0G prize tracks simultaneously.

The body of the document is organized as six parts: (1) NameWrapper deep dive, (2) tokenomics options and recommendation, (3) Solidity scaffolding, (4) ETHGlobal winning strategy, (5) ecosystem integration map, (6) KeeperHub integration. A prioritized build order closes the report.

Part 1 — ENS NameWrapper architecture for `bankon.eth`

Mainnet topology and the wrapping recipe

The NameWrapper at `0xD4416b13d2b3a9aBae7AcD5D6C2BbDBE25686401` is the canonical contract every wrapped name flows through. The ENS Registry (`0x00…2e1e`), BaseRegistrar (`0x57f1…eA85`), latest PublicResolver (`0x231b…8E63`), and the DAO-owned Universal Resolver proxy (`0xeEeE…EeEe`) round out the surface area the registrar will interact with. The token-id math is identical for every wrapped name: `tokenId = uint256(namehash(name))`, where

`namehash("bankon.eth") = keccak256(ETH_NODE || keccak256("bankon"))`. This contrasts with the BaseRegistrar's ERC-721, which uses `uint256(labelhash)` — a frequent source of bugs.

The wrapping flow for `bankon.eth` proceeds in five concrete on-chain steps.

1. Transfer the unwrapped `.eth` 2LD into the NameWrapper using `BaseRegistrar.safeTransferFrom(owner, NameWrapper, uint256(labelhash), abi.encode(label, owner, fuses, resolver))`. The wrapper's `onERC721Received` handler decodes that calldata and mints the ERC-1155 to the encoded owner. This is cheaper than the two-call `setApprovalForAll`+`wrapETH2LD` path.
2. Deploy the `BankonSubnameRegistrar` contract.
3. Burn `CANNOT_UNWRAP` (`0x0001`) on `bankon.eth` itself with `setFuses(node, 0x0001)`. This is irreversible, and is the prerequisite for ever burning fuses on any child subname.
4. Call `setApprovalForAll(registrar, true)` on the wrapper so the registrar can mint subnames without the parent NFT changing hands.
5. Optionally lock the renewal manager via `approve(renewalManager, parentTokenId)` followed by `setFuses(parent, CANNOT_APPROVE)` for unruggable renewal economics — a pattern winning ENS hacks have used since the original Sublettuce project.

Subname minting uses one of two NameWrapper functions. **`setSubnodeRecord`** is the production primitive because it issues the ERC-1155, sets the resolver, and burns fuses in a single call: `setSubnodeRecord(parentNode, label, owner, resolver, ttl, fuses, expiry)`. The recommended fuse profile for an agent subname is **`PARENT_CANNOT_CONTROL | CANNOT_UNWRAP | CANNOT_TRANSFER | CAN_EXTEND_EXPIRY = 0x50005`**, which produces a Locked, soulbound, agent-renewable identity capped at the parent's expiry. If records should be frozen post-mint, add `CANNOT_SET_RESOLVER | CANNOT_SET_TTL | CANNOT_CREATE_SUBDOMAIN` to reach `0x5003D`. For the demo, soulbound is the right default because reputation continuity demands non-transferability; for premium tradable agent names mint with `0x10001` (PCC + CU only) and let the secondary market work.

The fuse state machine, explained for engineers

The ENS fuse system is a 32-bit bitfield with three states that matter:

- **Wrapped** (default; parent retains full control)
- **Emancipated** (`PCC` burned; parent cannot rug)
- **Locked** (`PCC | CU` both burned; owner can now burn additional owner-controlled fuses)

The state machine has a recursive rule that trips up most implementers: to burn any owner-controlled fuse on a node N you must first burn `CU` on N, and to burn `CU` you must first have

`PCC` burned, which only the parent can do. For a `.eth` 2LD this is bootstrapped automatically — `wrapETH2LD` auto-burns `IS_DOT_ETH | PARENT_CANNOT_CONTROL` so `bankon.eth` lands in Emancipated state immediately. Burning `CU` on the parent in step three above is what makes minting Locked children possible.

Owner-controlled fuses live in the low 16 bits: `CANNOT_UNWRAP (0x1)`, `CANNOT_BURN_FUSES (0x2)`, `CANNOT_TRANSFER (0x4)`, `CANNOT_SET_RESOLVER (0x8)`, `CANNOT_SET_TTL (0x10)`, `CANNOT_CREATE_SUBDOMAIN (0x20)`, `CANNOT_APPROVE (0x40)`. Parent-controlled fuses live in the high 16 bits: `PARENT_CANNOT_CONTROL (0x10000)`, `IS_DOT_ETH (0x20000, internal-only)`, `CAN_EXTEND_EXPIRY (0x40000)`.

Thirteen parent-controlled bits and nine owner-controlled bits remain unreserved — the registrar can repurpose these for app-specific badges (verified-agent, BONAFIDE-tier, ERC-8004-attested, x402-enabled, KeeperHub-bonded), which is a cheap on-chain signal that wins technical-depth points in judging.

Resolution standards: ENSIP-10, ENSIP-11, ENSIP-19, and CCIP-Read

Four standards govern how clients resolve `agent.bankon.eth` to addresses, text records, and chain-specific endpoints.

ENSIP-10 wildcard resolution (`IExtendedResolver`, interface ID `0x9061b923`) lets a single resolver answer for every subname under a parent without ever creating registry records — this is what NameStone, JustaName, and Coinbase's `cb.id` (over two million subnames) use. The client strips labels until it finds a resolver, checks `supportsInterface(0x9061b923)`, and if true calls `resolve(dnsEncodedName, originalCallData)` instead of the legacy methods.

ENSIP-11 defines per-EVM-chain `addr(node, coinType)` records, where `coinType = 0x80000000 | chainId` for EVM chains; Base is `0x80002105`, Optimism is `0x8000000A`, Arbitrum is `0x8000A4B1`. Algorand is non-EVM and uses its **SLIP-44 coin index 283 → coinType** `0x8000011B` directly, which is what a wallet calls when it sees an Algorand-bound agent subname.

ENSIP-19 brings reverse resolution to L2s — agents on Base set their primary name via `L2ReverseRegistrar.setName("agent.bankon.eth")` on Base, and the L1 Universal Resolver fetches it via CCIP-Read.

CCIP-Read (EIP-3668) is the offchain escape hatch and the architectural fork in the road. A resolver reverts with `OffchainLookup(sender, urls, callData, callbackFunction, extraData)`; the client (any modern wallet, viem, ethers v6, ENSjs) substitutes `{sender}` and `{data}`, hits the gateway over HTTPS, and feeds the response back into the resolver's verification callback, which typically validates an EIP-712 signature from a known gateway operator. Trust models split into *fully trusted* (NameStone, JustaName — gateway signs, you trust the gateway operator) and *trustless* (Durin's Unruggable Gateway flavor — gateway returns a storage proof verified against an L1-posted state root).

For BANKON the cost arithmetic is stark: an onchain `setSubnodeRecord` mint plus three text records costs roughly 280k gas (~$14 at 20 gwei and $2.5k ETH), while a CCIP-Read offchain mint is essentially free. **At scale beyond ~500 agents the hybrid is the only sane answer**: locked onchain wrapped subnames for revenue-bearing premium agents, CCIP-Read offchain identifiers for the long tail.

Real-world registrar patterns and what to copy from each

- **Durin** (ENS Labs / NameStone-maintained at `github.com/namestonehq/durin`) is the L2-native onchain pattern — `L1Resolver` reverts CCIP-Read pointing to a Durin gateway that reads from an `L2Registry` ERC-721 you customize via `L2Registrar.register()`. The shared L1 resolver lives at `0x8A968aB9eb8C084FBC44c531058Fc9ef945c3D61`.
- **NameStone** is pure offchain CCIP-Read with a hosted DB and free tier; their SDK is `@namestone/namestone-sdk`.
- **Namespace.tech** is the most flexible commercial registrar — it supports L1 wrapped, L2 (Base), and offchain CCIP-Read with per-length pricing tiers configured by parent owners and a platform `mintFee` skim.
- **JustaName** is offchain-first with SIWE-authorized issuance and a strong React widget.
- **3DNS** is the closest tokenized-DNS comparable.

The official ENS reference implementations live at `github.com/ensdomains/ens-contracts/tree/feature/subdomain-registrar/contracts/subdomainregistrar` (`ForeverSubdomainRegistrar` and `RentalSubdomainRegistrar`) and are the templates `BankonSubnameRegistrar.sol` below directly extends.

The **strategically important context for 2026**: ENS Labs cancelled the Namechain L2 in February 2026, citing a 99% reduction in registration gas costs from Ethereum's 30M→60M gas-limit increase, and now ENSv2 deploys exclusively on L1 with a *hierarchical-registry* pattern where every `.eth` name gets its own registry contract. **This is the architecture BANKON is implementing**. Citing the ENSv2 design rationale and the ENS Labs November 2025 ERC-8004 blog post in the README aligns the submission with current sponsor priorities and demonstrates the team is reading ENS's strategic communications, which judges notice.

Part 2 — Tokenomics monetization deep dive

Comparable pricing landscape

Eleven naming and agent-identity systems benchmark cleanly against BANKON's design space. The numbers below are verified for Q1 2026.

System	Pricing model	3-char	4-char	5+ char	Renewal	Token
ENS .eth	Length-tiered, USD via Chainlink	$640/yr	$160/yr	$5/yr	Annual + 90-day grace + 21-day Dutch decay from $100M	ETH
NameStone	Issuer-defined; free API tier	n/a	n/a	n/a	Issuer-set	Any
Namespace.tech	Issuer-set length tiers + platform `mintFee`	varies	varies	varies	Inherits parent	ETH / L2 / offchain
JustaName	Free issuance via API	n/a	n/a	n/a	n/a	None
Lens v2 handle	Variable, has fluctuated $30–$200	market	market	market	One-time + transferable	GHO/MATIC historically
Farcaster fnames	**Free**, policy-gated reclaim	free	free	free	60-day idle reclaim; storage $7/yr separate	USDC for Pro tier
Unstoppable Domains	One-time, length-tiered	premium	$$	$5–$20	**Permanent**	Cards, ETH, USDC
Bonfida .sol	$20 USDC floor, FIDA buyback-and-burn	$20+	$20+	$20+	**Permanent**	USDC, FIDA
3DNS .box	$6/yr tokenization + ICANN; 2.5% royalty	TLD-dep	TLD-dep	TLD-dep	Annual	ETH/USDC on OP
Virtuals Protocol	100 $VIRTUAL launch + bonding	n/a	n/a	n/a	Token, not name	$VIRTUAL

System	Pricing model	3-char	4-char	5+ char	Renewal	Token
	curve to 42K graduation					
Olas service registry	Operator stake, no protocol fee	n/a	n/a	n/a	Service lifecycle	ETH/OLAS/custom
ERC-8004 Identity	Gas only; payments out-of-scope	n/a	n/a	n/a	Permanent NFT	Any
Friend.tech (curve ref)	`price = supply²/16,000` ETH; 5%+5% fee	n/a	n/a	n/a	Per trade	ETH on Base

Three observations drive the BANKON recommendation. First, **annual length-tiered rent is the dominant L1 model and the only one that has produced a serious treasury** — ENS DAO accumulated over 43,800 ETH lifetime by October 2024. Second, **agent-specific systems do not price by character length**; they price by economic role (Virtuals' bonding curve, Olas' bond, ERC-8004's gas-only mint). Third, **offchain registrars compete at $0 marginal cost**, which establishes the floor BANKON must justify pricing above.

Pricing-mechanism analysis

- **Flat fee** is too simple — it doesn't differentiate scarcity, encouraging squatting on premium short names.
- **Length-tiered** is battle-tested and strong, the ENS model, and what BANKON should adopt.
- **Character-class multipliers** (digit-only, alphanumeric, emoji, ticker-style) capture the "001.bankon.eth" or "$pyth.bankon.eth" demand and mirror domain-investor markets, but add governance overhead — a v2 add-on, not v1.
- **Bonding curves** are the wrong primitive for naming because each name has supply 1, so per-name curves degenerate; however, a **step bonding curve on total registry supply** (first 1,000 free, next 10,000 cheap, then standard tier) is a clean genesis-incentive mechanism that mirrors Virtuals' graduation logic.
- **Dutch auction** with exponential decay is correct for *expired* premium names and for genesis allocation of curated 3-character names — the .box launch ran a 6-day half-life decay from $7,680 to a $120 floor, ENS uses a 21-day decay from $100M to $0.

- **Harberger / partial common ownership** is structurally elegant against squatting but catastrophic for agent identity continuity (an agent's reputation is tied to its name; involuntary turnover destroys economic value), so reject it as the primary mechanism but offer it as opt-in mode for traders.
- **Subscription versus perpetual** comes down to whether you want recurring revenue and namespace cleanup; for autonomous agents that natively pay micropayments anyway, **annual rent auto-renewed via x402 is a perfect machine-payment use case** and the obvious right answer.
- **Reputation-gated free issuance** (Worldcoin-style, gated by BONAFIDE score or ERC-8004 TEE attestation) is the right tool for the long tail to compete with NameStone's free tier without becoming a spam vector.

The honest answer on **buyback-and-burn versus buyback-and-make**: Bonfida burns FIDA and Virtuals announced a $48M $VIRTUAL burn in January 2025, but Placeholder VC's Joel Monegro made a strong argument that buyback-and-make (redeploy into LP, staking, grants) funds growth where burn just deflates. For PYTHAI, which has no fixed peg and a small float, **buyback-and-make is the better model** — it funds the agent ecosystem (subsidizing free registrations for verified agents, paying x402 facilitator fees, seeding LP) rather than purely contracting supply.

Multi-token payment routing and x402 integration

The x402 protocol (Coinbase-led, now jointly stewarded with Cloudflare via the x402 Foundation) revives HTTP 402 as a generic micropayment standard. The wire flow is four steps:

1. Client GET → server returns `402 Payment Required` with a JSON `paymentRequirements` array listing accepted `(scheme, network, asset, amount, payTo, maxTimeoutSeconds)` tuples
2. Client constructs a signed payload (EIP-3009 `transferWithAuthorization` for USDC/EURC, Permit2 for arbitrary ERC-20s, native ATG for Algorand ASAs)
3. Client retries with `X-PAYMENT: <base64>`
4. Server verifies and settles via facilitator → returns 200 plus `X-PAYMENT-RESPONSE`

CDP's hosted facilitator covers Base, Polygon, Arbitrum, World Chain, and Solana with 1,000 free transactions per month and $0.001 per transaction thereafter; **Algorand support merged in February 2026** via GoPlausible's facilitator. CAIP-2 chain IDs (`eip155:8453`, `solana:<genesisHash>`, `algorand:<genesisHash>`) identify the settlement network.

The **BANKON x402 integration design** routes payments through an HTTPS gateway in front of the Solidity registrar. An agent on `mindx.pythai.net` calls `GET https://registrar.bankon.eth/v1/register?label=foo&owner=0xAGENT&years=1`. The

gateway returns a 402 with three accepted payment routes — USDC on Base, PYTHAI ASA on Algorand, native ETH on L1 — each binding a `commitmentNonce` derived from the registration parameters. The agent signs the appropriate authorization. The gateway calls facilitator `/verify` then `/settle`. **Crucially**, settlement happens on the user's source chain (Base or Algorand) into BANKON treasury addresses there; an off-chain relayer with a delegated `gatewaySigner` key then signs an EIP-712 voucher binding `(label, owner, expiry, paymentReceiptHash)` and the L1 registrar verifies that voucher in `register()`, marking the receipt used to prevent replay. This avoids the latency and cost of CCTP/LayerZero bridging on the registration hot path; treasury rebalancing happens out-of-band. For trust-minimized variants, deploy a Durin-style L2 registrar on Base so Base-originating agents stay on Base end-to-end and resolve via L1 CCIP-Read.

PYTHAI/PAI/THRUST/PAIMINT pricing requires USD-denominated targets translated to token amounts at quote time using a Uniswap v3 30-minute TWAP; the 402 response includes `validBefore = now + 60s` to bound oracle drift. A **20% native-PYTHAI discount** is the recommended incentive — large enough to drive token velocity, small enough to preserve revenue. THRUST/PAIMINT route through the same TWAP oracle; PAI uses a fixed-table fallback if liquidity is thin.

Revenue distribution and value accrual

The recommended split routes:

- **40%** to the **BANKON DAO multisig treasury** (analogous to wallet.ensdao.eth)
- **25%** to **PYTHAI buyback-and-make** (treasury-owned LP positions in the PYTHAI/USDC pool)
- **15%** to a **public-goods fund** seeding ERC-8004 ecosystem grants
- **10%** to **x402 facilitator and relayer operational costs**
- **10%** to a **squat-defense reserve** that pre-funds Dutch-auction starting prices for premium expired names

Governance over these splits and over the length-tier prices uses **vePYTHAI gauges** — Curve-style locked-token voting weighted by lock duration — which encourages long-horizon alignment from the heaviest registrar users.

Agent-specific tokenomics innovations worth implementing

Five innovations differentiate BANKON from generic naming registrars:

1. **Reputation-bonded subnames** require agents to stake a BONAFIDE-weighted PYTHAI deposit that is slashable on misbehavior — this is functionally identical to Olas' service-registry bond pattern but anchored to ENS identity.

2. **Earnings-shared subnames** auto-route a configurable percentage of an agent's x402 income on `mindx.pythai.net` back to the BANKON treasury via a hook in the x402 facilitator response — this is the "subname-as-revenue-share" pattern that hasn't shipped anywhere yet and would be a strong narrative beat in the demo.
3. **Tiered agent classes** map AgenticPlace's Soul/Mind/Hands cognitive architecture onto pricing: Soul-tier executive agents get reserved 3-character names with curated allocation, Mind-tier reasoning agents (mindX) get standard 4-char pricing, Hands-tier worker agents get the long-tail $1/yr tier or free-with-reputation.
4. **Composable nested namespaces** — `agent.mindx.bankon.eth` — let a Mind-tier agent operate its own sub-registrar with revenue splitting up the tree to the parent, which is exactly the hierarchical-registry pattern ENSv2 is building toward.
5. **Bundle pricing** (subname + AlgoIDNFT mint + x402 endpoint registration + ERC-8004 identity in one transaction) eliminates the multi-step UX that hurts adoption of competing systems.

Anti-squatting and dispute resolution

A four-layer defense applies:

1. **Reservation list** for protected names (project tickers, Soul-tier roles, ENS-Labs-blocklisted brand names)
2. **Identity verification gating** via AlgoIDNFT or ERC-8004 TEE-attestation for free issuance
3. **Farcaster-style 60-day idle reclaim** of dormant subnames
4. **Post-expiry exponential Dutch auction** for the 3- and 4-char tiers only

Disputed names route through **BONAFIDE Censura/Senatus** with vePYTHAI-weighted voting and a 7-day timelock, with on-chain enforcement via `setChildFuses` (the parent retains the ability to clear `PCC` on contested names because we deliberately mint them with `PCC` set but `CU` not yet burned, leaving a dispute window before locking).

Three candidate pricing schedules and the recommendation

Tier	LOW (accessibility)	MEDIUM (recommended)	HIGH (ENS-strict)
3-char	$80/yr	**$320/yr**	$640/yr
4-char	$25/yr	**$80/yr**	$160/yr
5-char	$3/yr	**$5/yr**	$10/yr

Tier	LOW (accessibility)	MEDIUM (recommended)	HIGH (ENS-strict)
6-char	$1/yr	**$3/yr**	$5/yr
7+ char	free, gated	**$1/yr**	$3/yr
ERC-8004 mint	bundled	**bundled**	bundled
PYTHAI discount	30%	**20%**	15%
Premium decay	$10K → $0 over 14d	**$1M → $0 over 21d**	$100M → $0 over 21d
Free gate	none	**≥10K PYTHAI or ERC-8004 TEE attestation**	none

Adopt the MEDIUM schedule. The 5-char $5/yr matches ENS exactly, preserving familiar pricing for human users adopting BANKON. The 3-char $320/yr is half of ENS's $640 — accessible enough for serious operators yet expensive enough to deter squatting on a namespace of only ~17,576 alphanumeric possibilities. The 7+ char $1/yr keeps the long tail near-free without crossing into spam territory (Farcaster's $5 floor is what they explicitly named to deter bots; we go lower because we have BONAFIDE and ERC-8004 gating that Farcaster lacks). Bundling ERC-8004 identity by default makes BANKON the easy button for compliant agent identity, which is the exact narrative ENS Labs is publishing.

The two alternatives serve different optics. The **LOW schedule** wins purely on UX and breadth of adoption — it is the right answer if hackathon judging weights agent-count demonstrations. The **HIGH schedule** wins on protocol-revenue narrative and DAO-treasury growth — the right answer if judges weight long-term sustainability. The MEDIUM schedule is the compromise that reads coherent on all four judging criteria: hackathon optics (familiar to ENS, novel in agent-tier features), long-term sustainability (recurring annual revenue with treasury split), integration coherence (PYTHAI discount, BONAFIDE gating, x402 native), and agent UX (sub-cent micropayments via x402, no bridge friction).

Part 3 — Solidity contract scaffolding (Foundry)

Project structure follows Foundry conventions: `src/` for contracts, `test/` for Solidity tests, `script/` for deployment, `foundry.toml` configured with mainnet and Sepolia RPC endpoints. All contracts target Solidity 0.8.24 with `pragma abicoder v2` implicit. OpenZeppelin v5 is used for `AccessControl`, `ReentrancyGuard`, `EIP712`, `ECDSA`, `Pausable`, and `SafeERC20`.

Core interfaces

```solidity
```

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

interface INameWrapper {
    function ownerOf(uint256 id) external view returns (address);
    function getData(uint256 id) external view returns (address owner, uint32 fuses,
    function setSubnodeOwner(bytes32 parentNode, string calldata label, address owne
    function setSubnodeRecord(bytes32 parentNode, string calldata label, address own
    function setChildFuses(bytes32 parentNode, bytes32 labelhash, uint32 fuses, uint
    function setFuses(bytes32 node, uint16 ownerControlledFuses) external returns (u
    function setResolver(bytes32 node, address resolver) external;
    function extendExpiry(bytes32 parentNode, bytes32 labelhash, uint64 expiry) exte
    function isWrapped(bytes32 node) external view returns (bool);
    function setApprovalForAll(address operator, bool approved) external;
    function approve(address to, uint256 tokenId) external;
}

interface IPublicResolver {
    function setAddr(bytes32 node, address a) external;
    function setAddr(bytes32 node, uint256 coinType, bytes calldata a) external;
    function setText(bytes32 node, string calldata key, string calldata value) exter
    function setContenthash(bytes32 node, bytes calldata hash) external;
    function multicall(bytes[] calldata data) external returns (bytes[] memory);
}

interface IBankonPriceOracle {
    function priceUSD(string calldata label, uint256 durationYears) external view re
    function priceInToken(string calldata label, uint256 durationYears, address toke
}

interface IBankonReputationGate {
    function isEligibleForFree(address agent) external view returns (bool);
    function isEligibleForRegistration(address agent) external view returns (bool);
    function bonafideScore(address agent) external view returns (uint256);
}

interface IMindXBridge {
    function isAttestedAgent(address agent, bytes32 mindxAgentId) external view retu
    function getAgentEndpoint(bytes32 mindxAgentId) external view returns (string me
}

interface IIdentityRegistry8004 {
    function register(address agentWallet, string calldata agentURI) external return
```

```
    function setMetadata(uint256 agentId, bytes32 key, bytes calldata value) externa
}
```

BankonSubnameRegistrar.sol

```solidity
```

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

import {AccessControl} from "@openzeppelin/contracts/access/AccessControl.sol";
import {ReentrancyGuard} from "@openzeppelin/contracts/utils/ReentrancyGuard.sol";
import {Pausable} from "@openzeppelin/contracts/utils/Pausable.sol";
import {EIP712} from "@openzeppelin/contracts/utils/cryptography/EIP712.sol";
import {ECDSA} from "@openzeppelin/contracts/utils/cryptography/ECDSA.sol";
import {ERC1155Holder} from "@openzeppelin/contracts/token/ERC1155/utils/ERC1155Hold

import {INameWrapper, IPublicResolver, IBankonPriceOracle,
        IBankonReputationGate, IIdentityRegistry8004} from "./interfaces/IBankon.sol
import {BankonPaymentRouter} from "./BankonPaymentRouter.sol";

/// @title BankonSubnameRegistrar
/// @notice ENS NameWrapper-based registrar issuing agent subnames under bankon.eth
/// @dev Mainnet deployment; integrates BONAFIDE reputation, x402 payments via vouch
///      and bundles ERC-8004 identity mint per ENS Labs Nov 2025 guidance.
contract BankonSubnameRegistrar is AccessControl, ReentrancyGuard, Pausable, EIP712,
    using ECDSA for bytes32;

    // ---------- Roles ----------
    bytes32 public constant BANKON_OPS_ROLE        = keccak256("BANKON_OPS_ROLE");
    bytes32 public constant BONAFIDE_GOV_ROLE      = keccak256("BONAFIDE_GOV_ROLE");
    bytes32 public constant MINDX_OPERATOR_ROLE    = keccak256("MINDX_OPERATOR_ROLE"
    bytes32 public constant GATEWAY_SIGNER_ROLE    = keccak256("GATEWAY_SIGNER_ROLE"

    // ---------- Constants ----------
    bytes32 public constant BANKON_NODE = 0x0; // set in constructor: namehash("bank
    uint32  public constant DEFAULT_FUSES =
        0x10000 | 0x1 | 0x4 | 0x40000; // PCC | CU | CANNOT_TRANSFER | CAN_EXTEND_EX

    bytes32 private constant REGISTRATION_TYPEHASH = keccak256(
        "Registration(string label,address owner,uint64 expiry,bytes32 paymentReceip
    );

    // ---------- Immutable wiring ----------
    INameWrapper public immutable nameWrapper;
    IPublicResolver public immutable defaultResolver;
    bytes32 public immutable parentNode;
    BankonPaymentRouter public immutable paymentRouter;

    // ---------- Mutable wiring ----------
```

```
IBankonPriceOracle public priceOracle;
IBankonReputationGate public reputationGate;
IIdentityRegistry8004 public identityRegistry8004;

// ---------- State ----------
mapping(bytes32 => bool) public usedReceipts;
mapping(bytes32 => string) public labelOf; // node -> label, for indexing conven
bool public erc8004BundleEnabled = true;

// ---------- Events ----------
event SubnameRegistered(
    bytes32 indexed node,
    string label,
    address indexed owner,
    uint64 expiry,
    uint256 priceUSD6,
    address paymentToken,
    bytes32 paymentReceiptHash,
    uint256 erc8004AgentId
);
event SubnameRenewed(bytes32 indexed node, uint64 newExpiry, uint256 priceUSD6);
event ResolverUpdated(bytes32 indexed node, address resolver);
event PriceOracleUpdated(address oldOracle, address newOracle);
event ReputationGateUpdated(address oldGate, address newGate);

// ---------- Errors ----------
error LabelTooShort();
error LabelEmpty();
error ReceiptAlreadyUsed();
error VoucherExpired();
error InvalidGatewaySignature();
error NotEligible();
error LabelAlreadyTaken();
error InvalidExpiry();

constructor(
    address _nameWrapper,
    address _defaultResolver,
    bytes32 _parentNode,
    address _paymentRouter,
    address _priceOracle,
    address _reputationGate,
    address _identityRegistry8004,
    address _admin
```

```
) EIP712("BankonSubnameRegistrar", "1") {
    nameWrapper          = INameWrapper(_nameWrapper);
    defaultResolver      = IPublicResolver(_defaultResolver);
    parentNode           = _parentNode;
    paymentRouter        = BankonPaymentRouter(_paymentRouter);
    priceOracle          = IBankonPriceOracle(_priceOracle);
    reputationGate       = IBankonReputationGate(_reputationGate);
    identityRegistry8004 = IIdentityRegistry8004(_identityRegistry8004);

    _grantRole(DEFAULT_ADMIN_ROLE, _admin);
    _grantRole(BANKON_OPS_ROLE, _admin);
}

// ============================================================================
// Registration
// ============================================================================

/// @notice Register a new agent subname after offchain x402 payment.
/// @dev Caller is typically the gateway relayer; voucher binds payment to param
function register(
    string calldata label,
    address owner,
    uint64 expiry,
    bytes32 paymentReceiptHash,
    uint256 deadline,
    bytes calldata gatewaySig,
    AgentMetadata calldata meta
) external nonReentrant whenNotPaused returns (bytes32 node, uint256 agentId) {
    if (bytes(label).length == 0) revert LabelEmpty();
    if (bytes(label).length < 3) revert LabelTooShort();
    if (block.timestamp > deadline) revert VoucherExpired();
    if (usedReceipts[paymentReceiptHash]) revert ReceiptAlreadyUsed();
    if (!reputationGate.isEligibleForRegistration(owner)) revert NotEligible();

    // Verify EIP-712 voucher from authorized gateway signer.
    bytes32 digest = _hashTypedDataV4(keccak256(abi.encode(
        REGISTRATION_TYPEHASH,
        keccak256(bytes(label)),
        owner,
        expiry,
        paymentReceiptHash,
        deadline
    )));
    address signer = digest.recover(gatewaySig);
```

```
    if (!hasRole(GATEWAY_SIGNER_ROLE, signer)) revert InvalidGatewaySignature();

    usedReceipts[paymentReceiptHash] = true;

    // Cap expiry by parent.
    (, , uint64 parentExpiry) = nameWrapper.getData(uint256(parentNode));
    if (expiry > parentExpiry) expiry = parentExpiry;

    // Step 1: temp self-own so we can write resolver records authoritatively.
    nameWrapper.setSubnodeOwner(parentNode, label, address(this), 0, expiry);
    node = keccak256(abi.encodePacked(parentNode, keccak256(bytes(label))));

    // Step 2: write resolver records via multicall.
    _writeAgentRecords(node, owner, meta);

    // Step 3: final transfer to user with locked fuses.
    nameWrapper.setSubnodeRecord(
        parentNode, label, owner,
        address(defaultResolver), 0,
        DEFAULT_FUSES, expiry
    );

    labelOf[node] = label;

    // Step 4: bundle ERC-8004 identity mint.
    if (erc8004BundleEnabled && address(identityRegistry8004) != address(0)) {
        agentId = identityRegistry8004.register(owner, meta.agentURI);
        identityRegistry8004.setMetadata(agentId, bytes32("bankon.ensName"), byt
        identityRegistry8004.setMetadata(agentId, bytes32("bankon.subnameNode"),
    }

    uint256 priceUSD6 = priceOracle.priceUSD(label, _yearsFromExpiry(expiry));
    emit SubnameRegistered(node, label, owner, expiry, priceUSD6, address(0), pa
}

/// @notice Free registration path for reputation-eligible agents.
function registerFree(
    string calldata label,
    address owner,
    uint64 expiry,
    AgentMetadata calldata meta
) external nonReentrant whenNotPaused returns (bytes32 node, uint256 agentId) {
    if (bytes(label).length < 7) revert LabelTooShort(); // free tier = 7+ chars
    if (!reputationGate.isEligibleForFree(owner)) revert NotEligible();
```

```
    (, , uint64 parentExpiry) = nameWrapper.getData(uint256(parentNode));
    if (expiry > parentExpiry) expiry = parentExpiry;

    nameWrapper.setSubnodeOwner(parentNode, label, address(this), 0, expiry);
    node = keccak256(abi.encodePacked(parentNode, keccak256(bytes(label))));
    _writeAgentRecords(node, owner, meta);
    nameWrapper.setSubnodeRecord(parentNode, label, owner, address(defaultResolv
    labelOf[node] = label;

    if (erc8004BundleEnabled && address(identityRegistry8004) != address(0)) {
        agentId = identityRegistry8004.register(owner, meta.agentURI);
    }
    emit SubnameRegistered(node, label, owner, expiry, 0, address(0), bytes32(0)
}

/// @notice Renew (extend expiry) of an existing subname.
function renew(string calldata label, uint64 newExpiry, bytes32 paymentReceiptHa
    external
    nonReentrant
    whenNotPaused
{
    bytes32 labelhash = keccak256(bytes(label));
    bytes32 node = keccak256(abi.encodePacked(parentNode, labelhash));
    if (block.timestamp > deadline) revert VoucherExpired();
    if (usedReceipts[paymentReceiptHash]) revert ReceiptAlreadyUsed();

    bytes32 digest = _hashTypedDataV4(keccak256(abi.encode(
        keccak256("Renewal(string label,uint64 newExpiry,bytes32 paymentReceiptH
        keccak256(bytes(label)), newExpiry, paymentReceiptHash, deadline
    )));
    if (!hasRole(GATEWAY_SIGNER_ROLE, digest.recover(gatewaySig))) revert Invali
    usedReceipts[paymentReceiptHash] = true;

    uint64 actual = nameWrapper.extendExpiry(parentNode, labelhash, newExpiry);
    emit SubnameRenewed(node, actual, priceOracle.priceUSD(label, _yearsFromExpi
}

// ==========================================================================
// Internal helpers
// ==========================================================================

struct AgentMetadata {
    string agentURI;          // ERC-8004 agent card JSON URI
```

```
    string mindxEndpoint;     // https://mindx.pythai.net/agent/<id>
    string x402Endpoint;      // https://...
    string algoIDNftDID;      // did:algo:...
    string keeperhubWorkflow; // KeeperHub workflow ID (wf_...)
    string keeperhubOrg;      // KeeperHub org ID
    bytes  contenthash;       // ipfs content hash
    uint256 baseAddress;      // Base L2 address (coinType 0x80002105)
    uint256 algoCoinAddrLen;  // sentinel for ALGO addr (coinType 0x8000011B)
    bytes algoAddr;
}

function _writeAgentRecords(bytes32 node, address owner, AgentMetadata calldata
    bytes[] memory calls = new bytes[](9);
    calls[0] = abi.encodeCall(IPublicResolver.setAddr, (node, owner));
    calls[1] = abi.encodeCall(IPublicResolver.setText, (node, "url", meta.mindxE
    calls[2] = abi.encodeCall(IPublicResolver.setText, (node, "x402.endpoint", m
    calls[3] = abi.encodeCall(IPublicResolver.setText, (node, "algoid.did", meta
    calls[4] = abi.encodeCall(IPublicResolver.setText, (node, "agent.card", meta
    calls[5] = abi.encodeCall(IPublicResolver.setText, (node, "keeperhub.workflo
    calls[6] = abi.encodeCall(IPublicResolver.setText, (node, "keeperhub.org", m
    calls[7] = abi.encodeCall(IPublicResolver.setContenthash, (node, meta.conten
    // Algorand chain-specific addr (SLIP-44 coinType 283 = 0x8000011B)
    calls[8] = abi.encodeWithSignature("setAddr(bytes32,uint256,bytes)", node, u
    defaultResolver.multicall(calls);
}

function _yearsFromExpiry(uint64 expiry) internal view returns (uint256) {
    if (expiry <= block.timestamp) return 0;
    return (expiry - block.timestamp) / 365 days;
}

function _concat(string memory a, string memory b) internal pure returns (string
    return string(abi.encodePacked(a, b));
}

// ==========================================================================
// Admin
// ==========================================================================

function setPriceOracle(address _o) external onlyRole(BONAFIDE_GOV_ROLE) {
    emit PriceOracleUpdated(address(priceOracle), _o);
    priceOracle = IBankonPriceOracle(_o);
}
function setReputationGate(address _g) external onlyRole(BONAFIDE_GOV_ROLE) {
```

```
        emit ReputationGateUpdated(address(reputationGate), _g);
        reputationGate = IBankonReputationGate(_g);
    }
    function setErc8004Bundle(bool enabled) external onlyRole(BONAFIDE_GOV_ROLE) { e
    function pause() external onlyRole(BANKON_OPS_ROLE) { _pause(); }
    function unpause() external onlyRole(BANKON_OPS_ROLE) { _unpause(); }
}
```

BankonPriceOracle.sol

```solidity
```

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

import {AccessControl} from "@openzeppelin/contracts/access/AccessControl.sol";

interface AggregatorV3Interface {
    function latestRoundData() external view returns (uint80,int256,uint256,uint256,
    function decimals() external view returns (uint8);
}

interface IUniV3TwapOracle {
    function consult(address pool, uint32 secondsAgo) external view returns (int24 a
}

/// @notice ENS-aligned length-tiered USD pricing with PYTHAI native discount.
contract BankonPriceOracle is AccessControl {
    bytes32 public constant GOV_ROLE = keccak256("GOV_ROLE");

    // Length-tier annual prices in USD-6 (USDC decimals).
    uint256 public price3 = 320_000000;
    uint256 public price4 =  80_000000;
    uint256 public price5 =   5_000000;
    uint256 public price6 =   3_000000;
    uint256 public price7plus = 1_000000;

    uint16 public pythaiDiscountBps = 2000; // 20%

    address public ethUsdFeed;       // Chainlink ETH/USD on L1
    address public pythaiUsdcPool;   // Uniswap v3 PYTHAI/USDC pool
    IUniV3TwapOracle public twap;

    address public pythaiToken;
    address public usdc;
    address public weth;

    event PricesUpdated(uint256 p3,uint256 p4,uint256 p5,uint256 p6,uint256 p7);

    constructor(address admin) {
        _grantRole(DEFAULT_ADMIN_ROLE, admin);
        _grantRole(GOV_ROLE, admin);
    }

    function priceUSD(string calldata label, uint256 durationYears) external view re
```

```
    uint256 perYear = _perYearUSD(bytes(label).length);
    return perYear * (durationYears == 0 ? 1 : durationYears);
}

function priceInToken(string calldata label, uint256 durationYears, address toke
    uint256 usd6 = _perYearUSD(bytes(label).length) * (durationYears == 0 ? 1 :
    if (token == usdc) return usd6;
    if (token == weth) return _usdToEth(usd6);
    if (token == pythaiToken) {
        uint256 discounted = usd6 * (10000 - pythaiDiscountBps) / 10000;
        return _usdToPythai(discounted);
    }
    revert("unsupported token");
}

function _perYearUSD(uint256 len) internal view returns (uint256) {
    if (len <= 3) return price3;
    if (len == 4) return price4;
    if (len == 5) return price5;
    if (len == 6) return price6;
    return price7plus;
}

function _usdToEth(uint256 usd6) internal view returns (uint256 wei_) {
    (,int256 px,,,) = AggregatorV3Interface(ethUsdFeed).latestRoundData();
    require(px > 0, "bad eth price");
    // px has 8 decimals; usd6 has 6 decimals; result in wei (18 decimals).
    return (usd6 * 1e20) / uint256(px);
}

function _usdToPythai(uint256 usd6) internal view returns (uint256) {
    // Read 30-min TWAP of PYTHAI/USDC pool. Implementation depends on pool toke
    int24 tick = twap.consult(pythaiUsdcPool, 1800);
    tick; // silence unused
    return usd6 * 50; // stub: 1 USDC = 50 PYTHAI, replace with real OracleLibra
}

// Admin
function setLengthPrices(uint256 p3,uint256 p4,uint256 p5,uint256 p6,uint256 p7)
    price3=p3; price4=p4; price5=p5; price6=p6; price7plus=p7;
    emit PricesUpdated(p3,p4,p5,p6,p7);
}
function setPythaiDiscount(uint16 bps) external onlyRole(GOV_ROLE) { require(bps
function setFeeds(address _eth,address _twap,address _pool,address _pythai,addre
```

```
        external onlyRole(GOV_ROLE)
    {
        ethUsdFeed=_eth; twap=IUniV3TwapOracle(_twap); pythaiUsdcPool=_pool;
        pythaiToken=_pythai; usdc=_usdc; weth=_weth;
    }
}
```

BankonPaymentRouter.sol

solidity

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

import {AccessControl} from "@openzeppelin/contracts/access/AccessControl.sol";
import {SafeERC20, IERC20} from "@openzeppelin/contracts/token/ERC20/utils/SafeERC20
import {ReentrancyGuard} from "@openzeppelin/contracts/utils/ReentrancyGuard.sol";

/// @notice Multi-token payment acceptance with x402 voucher hook.
/// @dev Payments may originate on Base/Algorand and be settled there; this
///      router only handles direct EVM-side ERC20/ETH receipts and emits
///      receipts that the registrar's gateway voucher binds against.
contract BankonPaymentRouter is AccessControl, ReentrancyGuard {
    using SafeERC20 for IERC20;

    bytes32 public constant TREASURER_ROLE = keccak256("TREASURER_ROLE");
    bytes32 public constant X402_FACILITATOR_ROLE = keccak256("X402_FACILITATOR_ROLE

    address public treasury;
    address public buybackVault;     // PYTHAI buyback-and-make vault (executed via
    address public publicGoodsFund;
    address public opsFund;
    address public squatReserve;

    uint16 public splitTreasuryBps    = 4000; // 40
    uint16 public splitBuybackBps     = 2500; // 25
    uint16 public splitPublicGoodsBps = 1500; // 15
    uint16 public splitOpsBps         = 1000; // 10
    uint16 public splitSquatBps       = 1000; // 10

    mapping(address => bool) public acceptedToken;

    event PaymentReceived(address indexed payer, address indexed token, uint256 amou
    event X402PaymentRecorded(bytes32 indexed receiptHash, address indexed network,
    event BuybackTriggerCrossed(uint256 vaultBalance, uint256 threshold);
    event SplitsUpdated(uint16 t,uint16 b,uint16 g,uint16 o,uint16 s);

    error UnsupportedToken();
    error InvalidSplits();

    constructor(address admin, address _treasury) {
        _grantRole(DEFAULT_ADMIN_ROLE, admin);
        _grantRole(TREASURER_ROLE, admin);
        treasury = _treasury;
```

```
}

/// @notice Direct EVM payment in an accepted ERC-20.
function payERC20(address token, uint256 amount, string calldata memo)
    external
    nonReentrant
    returns (bytes32 receiptHash)
{
    if (!acceptedToken[token]) revert UnsupportedToken();
    IERC20(token).safeTransferFrom(msg.sender, address(this), amount);
    receiptHash = keccak256(abi.encode(msg.sender, token, amount, block.number,
    _split(token, amount);
    emit PaymentReceived(msg.sender, token, amount, receiptHash, memo);
}

/// @notice Native ETH payment.
function payETH(string calldata memo) external payable nonReentrant returns (byt
    receiptHash = keccak256(abi.encode(msg.sender, address(0), msg.value, block.
    _splitETH(msg.value);
    emit PaymentReceived(msg.sender, address(0), msg.value, receiptHash, memo);
}

/// @notice Called by the x402 gateway facilitator after off-chain settlement
///         on Base/Algorand. Emits a receipt the registrar voucher binds to.
function recordX402Payment(
    address network, address asset, uint256 amount, string calldata caip2, bytes
) external onlyRole(X402_FACILITATOR_ROLE) {
    emit X402PaymentRecorded(receiptHash, network, asset, amount, caip2);
}

function _split(address token, uint256 amount) internal {
    uint256 t = amount * splitTreasuryBps / 10000;
    uint256 b = amount * splitBuybackBps / 10000;
    uint256 g = amount * splitPublicGoodsBps / 10000;
    uint256 o = amount * splitOpsBps / 10000;
    uint256 s = amount - t - b - g - o;
    IERC20(token).safeTransfer(treasury, t);
    IERC20(token).safeTransfer(buybackVault, b);
    IERC20(token).safeTransfer(publicGoodsFund, g);
    IERC20(token).safeTransfer(opsFund, o);
    IERC20(token).safeTransfer(squatReserve, s);
}
function _splitETH(uint256 amount) internal {
    uint256 t = amount * splitTreasuryBps / 10000;
```

```
        uint256 b = amount * splitBuybackBps / 10000;
        uint256 g = amount * splitPublicGoodsBps / 10000;
        uint256 o = amount * splitOpsBps / 10000;
        uint256 s = amount - t - b - g - o;
        (bool ok1,) = treasury.call{value:t}(""); require(ok1);
        (bool ok2,) = buybackVault.call{value:b}(""); require(ok2);
        (bool ok3,) = publicGoodsFund.call{value:g}(""); require(ok3);
        (bool ok4,) = opsFund.call{value:o}(""); require(ok4);
        (bool ok5,) = squatReserve.call{value:s}(""); require(ok5);
    }

    // Admin
    function setAccepted(address token, bool ok) external onlyRole(TREASURER_ROLE) {
    function setRecipients(address _t,address _b,address _g,address _o,address _s) e
        treasury=_t; buybackVault=_b; publicGoodsFund=_g; opsFund=_o; squatReserve=_
    }
    function setSplits(uint16 t,uint16 b,uint16 g,uint16 o,uint16 s) external onlyRo
        if (t+b+g+o+s != 10000) revert InvalidSplits();
        splitTreasuryBps=t; splitBuybackBps=b; splitPublicGoodsBps=g; splitOpsBps=o;
        emit SplitsUpdated(t,b,g,o,s);
    }
}
```

BankonReputationGate.sol

```solidity
```

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

import {AccessControl} from "@openzeppelin/contracts/access/AccessControl.sol";

interface IBONAFIDE {
    function score(address) external view returns (uint256);
    function isCensured(address) external view returns (bool);
}
interface IERC8004Identity {
    function balanceOf(address owner) external view returns (uint256);
    function isTeeAttested(address owner) external view returns (bool);
}
interface IERC20Min { function balanceOf(address) external view returns (uint256); }

contract BankonReputationGate is AccessControl {
    bytes32 public constant GOV_ROLE = keccak256("GOV_ROLE");

    IBONAFIDE public bonafide;
    IERC8004Identity public erc8004;
    IERC20Min public pythai;

    uint256 public minBonafideRegistration = 0;     // anyone can pay-to-play
    uint256 public minBonafideFreeTier     = 100;   // free for reputable agents
    uint256 public minPythaiHoldFreeTier   = 10_000 ether;

    constructor(address admin, address _bonafide, address _erc8004, address _pythai)
        _grantRole(DEFAULT_ADMIN_ROLE, admin);
        _grantRole(GOV_ROLE, admin);
        bonafide = IBONAFIDE(_bonafide);
        erc8004  = IERC8004Identity(_erc8004);
        pythai   = IERC20Min(_pythai);
    }

    function isEligibleForRegistration(address agent) external view returns (bool) {
        if (bonafide.isCensured(agent)) return false;
        return bonafide.score(agent) >= minBonafideRegistration;
    }

    function isEligibleForFree(address agent) external view returns (bool) {
        if (bonafide.isCensured(agent)) return false;
        bool teeOk = address(erc8004) != address(0) && erc8004.isTeeAttested(agent);
        bool stake = pythai.balanceOf(agent) >= minPythaiHoldFreeTier;
```

```
        bool rep   = bonafide.score(agent) >= minBonafideFreeTier;
        return teeOk || stake || rep;
    }

    function bonafideScore(address agent) external view returns (uint256) { return b

    function setThresholds(uint256 _reg, uint256 _free, uint256 _stake) external onl
        minBonafideRegistration = _reg; minBonafideFreeTier = _free; minPythaiHoldFr
    }
    function setSources(address _b,address _e,address _p) external onlyRole(GOV_ROLE
        bonafide=IBONAFIDE(_b); erc8004=IERC8004Identity(_e); pythai=IERC20Min(_p);
    }
}
```

Foundry test scaffolding

```solidity
```

```
// test/BankonSubnameRegistrar.t.sol
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

import "forge-std/Test.sol";
import "../src/BankonSubnameRegistrar.sol";
import "../src/BankonPriceOracle.sol";
import "../src/BankonPaymentRouter.sol";
import "../src/BankonReputationGate.sol";

contract BankonRegistrarMainnetForkTest is Test {
    address constant NAME_WRAPPER     = 0xD4416b13d2b3a9aBae7AcD5D6C2BbDBE25686401;
    address constant PUBLIC_RESOLVER  = 0x231b0Ee14048e9dCcD1d247744d114a4EB5E8E63;
    address constant BANKON_OWNER     = address(0xBEEF);
    bytes32 constant BANKON_NODE      = 0x0; // set in setUp via namehash

    BankonSubnameRegistrar registrar;
    BankonPriceOracle oracle;
    BankonPaymentRouter router;
    BankonReputationGate gate;
    address gatewaySigner; uint256 gatewayKey;

    function setUp() public {
        vm.createSelectFork(vm.rpcUrl("mainnet"));
        (gatewaySigner, gatewayKey) = makeAddrAndKey("gateway");
        // ... deploy BONAFIDE / ERC8004 mocks, oracle, gate, router, registrar
        // ... wrap bankon.eth in fork via prank of real owner
        // ... approveForAll(registrar)
        // ... grant GATEWAY_SIGNER_ROLE to gatewaySigner
    }

    function test_Register_HappyPath() public {
        BankonSubnameRegistrar.AgentMetadata memory meta = BankonSubnameRegistrar.Ag
            agentURI: "ipfs://bafy.../agent.json",
            mindxEndpoint: "https://mindx.pythai.net/agent/test",
            x402Endpoint: "https://x402.bankon.eth/test",
            algoIDNftDID: "did:algo:abc",
            keeperhubWorkflow: "wf_test123",
            keeperhubOrg: "org_bankon",
            contenthash: hex"e30101701220",
            baseAddress: 0,
            algoCoinAddrLen: 32,
            algoAddr: hex"0123456789abcdef0123456789abcdef0123456789abcdef012345678
```

```
    });

    uint64 expiry = uint64(block.timestamp + 365 days);
    bytes32 receiptHash = keccak256("receipt-1");
    uint256 deadline = block.timestamp + 1 hours;

    bytes32 digest = registrar.hashRegistration("test-agent", address(this), exp
    (uint8 v, bytes32 r, bytes32 s) = vm.sign(gatewayKey, digest);
    bytes memory sig = abi.encodePacked(r, s, v);

    (bytes32 node, uint256 agentId) = registrar.register(
        "test-agent", address(this), expiry, receiptHash, deadline, sig, meta
    );

    assertEq(INameWrapper(NAME_WRAPPER).ownerOf(uint256(node)), address(this));
    (, uint32 fuses,) = INameWrapper(NAME_WRAPPER).getData(uint256(node));
    assertEq(fuses & 0x10000, 0x10000); // PCC burned
    assertEq(fuses & 0x1, 0x1);          // CU burned
    assertEq(fuses & 0x4, 0x4);          // CANNOT_TRANSFER burned
}

function testFuzz_Register_LabelLength(uint8 len) public {
    len = uint8(bound(len, 3, 50));
    // ... build label of length len, register, assert success
}

function test_Register_RejectsReplay() public {
    // ... call register twice with same receiptHash, expect ReceiptAlreadyUsed
}

function test_Register_RejectsBadSignature() public {
    // ... sign with non-gateway key, expect InvalidGatewaySignature
}

function test_Register_FreeTier_RequiresEligibility() public {
    // ... gate returns false, expect NotEligible
}

function test_Renew_ExtendsExpiry() public { /* ... */ }

function testFuzz_Pricing_LengthTiers(uint8 len, uint8 years_) public {
    len = uint8(bound(len, 3, 30));
    years_ = uint8(bound(years_, 1, 10));
    // assert oracle pricing follows the tier curve and scales linearly with yea
```

```
    }
}
```

Foundry deployment script

```solidity
```

```
// script/Deploy.s.sol
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

import "forge-std/Script.sol";
import "../src/BankonSubnameRegistrar.sol";
import "../src/BankonPriceOracle.sol";
import "../src/BankonPaymentRouter.sol";
import "../src/BankonReputationGate.sol";

contract DeployBankon is Script {
    address constant NAME_WRAPPER_MAINNET = 0xD4416b13d2b3a9aBae7AcD5D6C2BbDBE256864
    address constant PUBLIC_RESOLVER_MAINNET = 0x231b0Ee14048e9dCcD1d247744d114a4EB5

    function run() external {
        uint256 pk = vm.envUint("DEPLOYER_PK");
        address admin = vm.envAddress("ADMIN");
        address bonafide = vm.envAddress("BONAFIDE");
        address erc8004Identity = vm.envAddress("ERC8004_IDENTITY");
        address pythai = vm.envAddress("PYTHAI");
        bytes32 parentNode = vm.envBytes32("BANKON_NODE");
        address nw = block.chainid == 1 ? NAME_WRAPPER_MAINNET : vm.envAddress("NAME
        address pr = block.chainid == 1 ? PUBLIC_RESOLVER_MAINNET : vm.envAddress("P

        vm.startBroadcast(pk);

        BankonPriceOracle oracle = new BankonPriceOracle(admin);
        BankonPaymentRouter router = new BankonPaymentRouter(admin, vm.envAddress("T
        BankonReputationGate gate = new BankonReputationGate(admin, bonafide, erc800

        BankonSubnameRegistrar registrar = new BankonSubnameRegistrar(
            nw, pr, parentNode,
            address(router), address(oracle), address(gate),
            erc8004Identity, admin
        );

        console2.log("Oracle  :", address(oracle));
        console2.log("Router  :", address(router));
        console2.log("Gate    :", address(gate));
        console2.log("Registrar:", address(registrar));

        vm.stopBroadcast();
```

```
    }
}
```

Resolver records emitted

The registrar writes nine canonical resolver records on every mint, indexable via The Graph for the demo subgraph: `addr` (owner), `text("url")` (mindX endpoint), `text("x402.endpoint")`, `text("algoid.did")`, `text("agent.card")` (ERC-8004 agent-card URI), `text("keeperhub.workflow")`, `text("keeperhub.org")`, `contenthash` (IPFS), and `addr(node, 0x8000011B)` (ENSIP-11 Algorand address). The two `SubnameRegistered` and `SubnameRenewed` events plus the resolver's standard `AddrChanged`/`TextChanged`/`ContenthashChanged` give a complete subgraph indexing surface.

Part 4 — ETHGlobal OpenAgents winning strategy

Verbatim prize criteria — ENS

The ENS sponsor allocates **$5,000 across two parallel $2,500 sub-tracks** at OpenAgents (Apr 24 – May 6, 2026).

Track A — Best ENS Integration for AI Agents ($1,250 / $750 / $500): *"AI agents need persistent, human-readable identities too. Build a functional project where ENS is the identity mechanism for one or more AI agents. ENS should be doing real work — resolving the agent's address, storing its metadata, gating access, enabling discovery, or coordinating agent-to-agent interaction."*

Track B — Most Creative Use of ENS ($1,250 / $750 / $500): *"Store verifiable credentials or zk proofs in text records. Build privacy features with auto-rotating addresses on each resolution. Use subnames as access tokens. Surprise us!"*

The qualification clause shared across both tracks is dispositive: *"ENS should clearly improve the product. Demo must be functional (no hard-coded values). Submit with a video or live demo link."* No mandate on NameWrapper or CCIP-Read or any L2; the rubric weights *real work* and *functional non-hardcoded* demos.

The strategic insight is that **BANKON qualifies for both ENS sub-tracks simultaneously** — it is canonically agent-identity (Track A) and uses subnames in unusual ways (subname as x402 paywall, AlgoIDNFT credential pointer, BONAFIDE reputation anchor, KeeperHub workflow discovery → Track B's "subnames as access tokens / verifiable credentials in text records"). Submitting once counts as one Partner Prize selection and is eligible for both.

Composable bounties at OpenAgents

The submission can select up to three Partner Prizes. The recommended stack is **ENS +**

KeeperHub + 0G, addressing $25k of prize pool.

- **KeeperHub ($5,000)** explicitly names x402 in its Focus Area 2: *"Integrate KeeperHub with payment rails like x402 or MPP. Show how agents can pay for services, settle transactions, or route payment flows into KeeperHub execution."* BANKON's x402 layer is a direct fit. (See Part 6 for the full integration plan.)
- **0G ($15,000 split across $7,500 Framework + $7,500 iNFT)** rewards persisting agent state in 0G Storage and minting agents as ERC-7857 iNFTs — every BANKON subname becomes an iNFT with embedded mindX state.
- **Gensyn AXL ($5,000)** is an alternative third pick if the demo centers on agent-to-agent communication; it requires demonstrating two separate AXL nodes communicating, which slots cleanly into the demo's "second agent calls the first via x402" beat.

Notably, **no standalone x402, Coinbase, Base, ERC-8004, or Algorand bounty exists at OpenAgents** — these compose only narratively or via KeeperHub Focus Area 2. ERC-8004 alignment remains a strong scoring signal because of the ENS Labs November 2025 blog post explicitly endorsing the ENS + ERC-8004 + x402 stack.

Patterns that win ENS tracks

Synthesis from the past four ETHGlobal events with ENS sponsorship (Cannes 2026's ENShell, Taipei 2025's ENSpin which won $2,500 first place, NYC 2025's AgentArena and Autonome and "Verified AI agent marketplace," and the NameWrapper-era Sublettuce and Immutable ENS Websites) yields seven explicit patterns:

1. **ENS does real work, not branding** — ENS's repeated phrase across briefs is *"It should be obvious how ENS improves your product and is not just implemented as an afterthought."*
2. **Programmatic subname issuance during the demo, not a link to app.ens.domains**; Istanbul rules disqualified the latter explicitly.
3. **NameWrapper sophistication is rewarded** — Sublettuce, Immutable ENS Websites, and AgentArena all leveraged fuses to enforce trustless properties.
4. **Text records as a programmable schema**, storing verifiable config (prompt hash, model, payment endpoint, credential, reputation pointer, ZK proofs) — this is *literally quoted* in Track B's brief.
5. **Functional demos with no hardcoded values**, which the rubric singles out twice.
6. **Vertical clarity plus composability** — winners pair ENS with a strong second narrative (agents, payments, privacy, DeFi); BANKON has agents + payments + reputation + execution, four vertical anchors.

7. **Live onchain transactions during the demo video** — winning videos almost always show MetaMask popping for a registration tx and the subname appearing on app.ens.domains.

The 4-minute demo flow

The demo runs 3:55, well under 5 minutes, matching ETHGlobal's preference for tight pitches.

Time	Beat	Screen	Voiceover
0:00–0:20	Cold open	Split screen: `0xa83c…f41` vs `arb-trader.bankon.eth`	*"AI agents have wallets but no names, no reputation, and no way to charge each other. We fix that in one contract."*
0:20–0:50	Architecture	Animated stack: mindX → BANKON Registrar → NameWrapper → text records (x402, AlgoIDNFT, BONAFIDE, mindX manifest, KeeperHub workflow)	*"BANKON is an ENS subname registrar built on NameWrapper. Every mindX agent gets a subname under bankon.eth — its passport, API endpoint, credit score, bank account, and execution surface."*
0:50–1:30	Live mint	Terminal `mindx spawn --name arb-trader` → MetaMask pop → tx confirms → app.ens.domains shows the records populated	*"Spawning a new agent. mindX calls our registrar's register() which calls setSubnodeRecord, burns PARENT_CANNOT_CONTROL so the agent can never be rugged, and writes nine resolver records in one tx."*
1:30–2:00	Resolver readout	ENS app: x402 endpoint, AlgoIDNFT DID, BONAFIDE score, KeeperHub workflow ID, mindX manifest CID	*"Any other agent can resolve arb-trader.bankon.eth and instantly know how to pay it, what credentials it holds, how trustworthy it is, and which onchain workflow it exposes."*
2:00–2:30	A2A x402 call via KeeperHub	Second terminal: Claude Code with KeeperHub MCP → resolves `data-oracle.bankon.eth` → reads `keeperhub.workflow` → `execute_workflow` over MCP → x402 settles → tx hash returns	*"arb-trader pays another agent for data. mindX resolves the subname, reads its KeeperHub workflow ID, executes it via MCP, pays a penny in USDC via x402. KeeperHub guarantees the onchain leg lands. This is the prize sponsor's brief verbatim."*
2:30–2:50	Reputation update with KeeperHub-guaranteed slashing path	Etherscan: BONAFIDE registry update → ENS text record reflects new score via CCIP-Read; KeeperHub dashboard shows the slashing workflow armed	*"Successful call → BONAFIDE bumps the reputation. If the agent misbehaves, the BankonReputationKeeper workflow fires the slash automatically. ENS Labs called for ENS + ERC-8004 + x402 in November. We added KeeperHub for execution."*
2:50–3:20	KeeperHub buyback	Dashboard shows running `PYTHAIBuybackKeeper`	*"40% to treasury, 25% to KeeperHub-executed buyback-and-make. Economics*

Time	Beat	Screen	Voiceover
	workflow	workflow on a cron trigger	*are not just split — they're enforced."*
3:20–3:40	Revenue flow	Dashboard: subname fee → bankon.eth treasury; x402 fees → owner; renewal stream visible	*"Registrar fees fund the parent forever. Subnames are unruggable — fuses burned at mint."*
3:40–3:55	Close	Logos: ENS, KeeperHub, 0G	*"BANKON: the ENS-native registrar for the agent economy. Live on Sepolia. github.com/bankon — thanks ETHGlobal."*

Risk factors and disqualification triggers

Ten failure modes lose ENS tracks: cosmetic ENS use; hardcoded demo values; no live tx during judging; subname issuance via ENS app rather than your contract; over-claiming "AI agent" without an agent loop; trying to win every bounty and integrating none deeply; missing FEEDBACK.md / video / public repo; centralized infra masquerading as decentralized; single commit / no version history (ETHGlobal disqualifies); pre-existing project rebadged.

The mitigation for the last is critical — BANKON exists as a brand, but the registrar contract, mindX integration, x402 plumbing, and KeeperHub bridge must be net-new during the hackathon, with commit history proving it. Document AI-tool usage (Cursor/Claude Code/Copilot) per ETHGlobal disclosure rules.

One-line pitch (mirrors ENS's own copy)

"BANKON is the ENS subname registrar for the agent economy: it gives every mindX agent a name, a reputation, a place to be found, and a guaranteed onchain execution surface — and lets agents pay each other through that name via x402."

Part 5 — Integration touchpoints map

End-to-end agent lifecycle sequence

The full flow from agent spawn to revenue-flowing reputation-anchored ENS identity follows a nine-step sequence:

1. A developer or another agent calls `POST https://mindx.pythai.net/agent/spawn` with a manifest (name, capabilities, model, prompt hash). The mindX runtime allocates a `mindxAgentId`, provisions an agent wallet (custodial or via Parsec/agentic-wallet

delegation), and returns an attestation that the BANKON registrar's `IMindXBridge` will later verify.

2. mindX optionally mints an AlgoIDNFT — an Algorand ASA with `freeze=creator`, `clawback=""` blanked at mint, manager retained — producing a `did:algo:<asaId>` DID.
3. mindX provisions a KeeperHub workflow for the agent (renewal automation by default, plus any agent-specific workflows declared in the manifest). The KeeperHub workflow ID is returned for inclusion in the resolver records.
4. mindX calls the BANKON gateway at `https://registrar.bankon.eth/v1/register` with `{label, owner, years, agentMetadata}`.
5. Gateway returns 402 with three accepted payment routes (USDC on Base, PYTHAI ASA on Algorand, ETH on L1), each binding a `commitmentNonce`.
6. Agent signs the appropriate authorization (EIP-3009 or Algorand ATG) and retries.
7. Gateway verifies and settles via CDP/GoPlausible facilitator on the source chain, then signs an EIP-712 voucher binding `(label, owner, expiry, paymentReceiptHash, deadline)`.
8. Relayer calls `BankonSubnameRegistrar.register()` on L1 with the voucher; the contract verifies the gateway signature, marks the receipt used, executes the temp-self-own → write-records → final-transfer-with-locked-fuses dance against NameWrapper, and bundles the ERC-8004 Identity NFT mint.
9. The registrar writes a BONAFIDE seed score (default tier or carried over from any prior reputation) and emits `SubnameRegistered`.

The agent now has a discoverable identity at `agent.bankon.eth` with nine resolver records pointing to its mindX endpoint, x402 endpoint, AlgoIDNFT DID, agent-card JSON, IPFS contenthash, KeeperHub workflow, KeeperHub org, and Algorand address.

The chainmapping question (`allchain.html`)

The `allchain.html` page on `agenticplace.pythai.net` could not be retrieved during research because all `pythai.net` subdomains return permission errors to public fetchers and no archive snapshot exists. The user must paste this page's contents into the design loop to enable enumeration of which chains, tokens, and contracts integrate.

Pending that, the architectural assumption is that BANKON treats `allchain.html` as the authoritative chain registry — each chain entry there gets a corresponding `addr(node, coinType)` resolver record per ENSIP-11 (or SLIP-44 coinType for non-EVM chains like Algorand 0x8000011B). The registrar's `_writeAgentRecords` helper is structured to accept arbitrary chain-coinType-address tuples; production should read the `allchain.html` registry at agent-spawn time and write all relevant chain records in one multicall.

API endpoints needed on bankon.pythai.net

The BANKON gateway service exposes seven endpoints:

- `POST /v1/register/quote` returns 402 with payment options for a label and duration (no state change).
- `POST /v1/register` accepts `X-PAYMENT` and a registration body, verifies and settles the payment, signs the EIP-712 voucher, and dispatches the L1 relayer call; returns 200 with the L1 tx hash and eventually the wrapped NFT tokenId.
- `POST /v1/renew` mirrors register for the renew flow.
- `GET /v1/availability/:label` checks NameWrapper for collision.
- `GET /v1/agent/:label` returns the resolved profile (calls Universal Resolver + ENS subgraph).
- `POST /v1/x402/callback` is the facilitator webhook endpoint.
- `GET /v1/treasury` returns split balances and recent splits for transparency.

The mindX side calls `POST /v1/register` directly during agent spawn; the AgenticPlace marketplace calls `GET /v1/agent/:label` for listing pages.

Foundry deployment plan

For the hackathon, deploy to **Sepolia** as the primary demo environment (low gas, ENS NameWrapper available, app.ens.domains supports Sepolia). The deployment sequence is:

1. Wrap a `bankon-test.eth` Sepolia name into NameWrapper.
2. Deploy `BankonPriceOracle`, `BankonPaymentRouter`, `BankonReputationGate`, `BankonSubnameRegistrar` via `forge script script/Deploy.s.sol --rpc-url $SEPOLIA --broadcast`.
3. Burn `CANNOT_UNWRAP` on the parent.
4. `setApprovalForAll(registrar, true)`.
5. Seed mock BONAFIDE/ERC-8004 contracts.
6. Verify on Etherscan with `forge verify-contract`.

Mainnet deployment uses the same script with chainId-conditional addresses and additional safety: a multisig admin, Tenderly simulation pre-broadcast, and a 7-day timelock on `setPriceOracle` and `setReputationGate` calls.

L2 deployment options: Base via the Durin pattern (L2 registry plus L1 CCIP-Read resolver) is the strongest option because it matches CDP x402 native settlement and ENS's own L2-name direction, even though ENSv2 stays on L1. Optimism and Arbitrum work identically via Durin.

CCIP-Read offchain alternative is reserved for the long tail of free-tier agents where onchain gas would be prohibitive — implement this as a v2 feature, not v1, because the ENS judges reward onchain depth more than scale.

Part 6 — KeeperHub integration

What KeeperHub is

KeeperHub is the **execution and reliability layer** for onchain agents — Turnkey-secured non-custodial wallets, multi-RPC failover, exponential backoff, nonce management, MEV-aware private routing, and a 30%-cheaper-than-baseline gas oracle, exposed through three surfaces:

- An **MCP server** (HTTP/SSE transport, available at `github.com/KeeperHub/keeperhub-mcp`)
- A **REST API** at `app.keeperhub.com`
- A **CLI**

It already powers Sky Protocol (formerly MakerDAO). Twelve EVM chains, 20+ protocol integrations (Aave, Spark, Lido, Safe, Morpho, Pendle, Compound, Yearn, Curve, Uniswap, CowSwap, Chronicle). Pay-per-execution in **USDC via x402 or MPP**.

The MCP server exposes tools including `list_workflows`, `get_workflow`, `create_workflow`, `update_workflow`, `delete_workflow`, `ai_generate_workflow`, `execute_workflow`, `get_execution_status`, `get_execution_logs`, `list_projects`, `list_tags`, and direct-execution primitives `execute_transfer`, `execute_contract_call`, `execute_check_and_execute`, `get_direct_execution_status`.

Prize breakdown

The pool is **$5,000 across two parallel tracks plus a feedback bounty**:

- **Best Use of KeeperHub**: $4,500 ranked ($2,500 / $1,500 / $500) split across:
  - **Focus Area 1**: innovative use of KeeperHub — any meaningful application
  - **Focus Area 2**: integration with payments (x402/MPP) or with agent frameworks (ElizaOS, LangChain, CrewAI, OpenClaw)
  - Best work wins regardless of track
- **Builder Feedback Bounty**: $500 (2 × $250) for actionable UX/bug/docs/feature feedback during integration

Why BANKON is a top-tier fit for KeeperHub Focus Area 2

The Focus Area 2 brief reads: *"Integrate KeeperHub with payment rails like x402 or MPP. Show how agents can pay for services, settle transactions, or route payment flows into KeeperHub execution."* BANKON already routes x402 payments through a multi-chain gateway. Wiring those payments to trigger KeeperHub-executed onchain operations is a one-page bridge that hits **both** ends of the brief: payments (x402) and execution (KeeperHub). And because BANKON is also competing for the ENS prize, KeeperHub becomes the *delivery mechanism* for several BANKON economic primitives that otherwise would be hand-waved in the demo.

Seven concrete integration points, ranked by leverage

The recommended integration is a **bidirectional bridge**: BANKON registrar publishes resolver records pointing to KeeperHub workflows, and KeeperHub workflows trigger BANKON registrar functions. Ordered by demo-impact:

1. **Agent execution surface as a resolver record (the killer feature).** Every BANKON subname gets a new resolver text record `keeperhub.workflow = wf_<id>`. When another agent resolves `data-oracle.bankon.eth`, it now discovers not just the agent's x402 endpoint but its **KeeperHub workflow ID**. The calling agent can invoke that workflow over MCP/REST, paying via x402, and KeeperHub guarantees the onchain leg lands. This is the single most narratively powerful demo beat — *"ENS resolves to a KeeperHub workflow; another agent pays x402 and the trade settles guaranteed."*
2. **Subname renewal automation.** A `BankonRenewalKeeper` workflow runs `cron: 0 0 * * *`, queries the registrar for subnames expiring within 30 days, and for each one with auto-renew enabled by the owner, calls the BANKON gateway's `/v1/renew` endpoint paying via x402. KeeperHub handles retries when gas spikes or RPC fails. Textbook KeeperHub recurring-operations workflow that addresses one of the registrar's hardest UX problems.
3. **PYTHAI buyback-and-make execution.** The 25% revenue split feeding `buybackVault` needs an actual onchain swap on Uniswap or CowSwap. A `PYTHAIBuybackKeeper` workflow triggers when the vault's USDC balance crosses a threshold, calls `cowswap.swap_exact_input`, and routes proceeds into a treasury-owned LP position. KeeperHub's CowSwap and Uniswap integrations are first-class.
4. **BONAFIDE reputation slashing.** When BONAFIDE Censura issues a slashing decision against a misbehaving agent, a KeeperHub `check_and_execute` workflow reads the BONAFIDE censure registry and, if the agent has a bonded subname, executes the slash

transaction (`registrar.slashBond(node, amount)`) reliably. This decouples governance from execution, mirroring how Sky Protocol uses KeeperHub today.

5. **Squat-defense Dutch auction price updates.** Premium expired subnames decay from $1M → $0 over 21 days. The price decay needs to be monotonic and frontrun-resistant; a KeeperHub workflow updates the on-chain Dutch auction state on a schedule (or reads-and-conditional-writes), guaranteeing the price ladder lands on every block.
6. **Cross-chain treasury rebalancing.** When x402 payments settle on Base or Algorand, a KeeperHub workflow watches treasury balances and triggers Across/CCTP bridge transactions back to L1 when balances cross thresholds. Exactly the *"route payment flows into KeeperHub execution"* phrase from the prize brief.
7. **Agent-to-agent settlement guarantee.** When `arb-trader.bankon.eth` pays `data-oracle.bankon.eth` for a price, the data oracle's response includes a payload the trader needs to act on (e.g., execute a swap on Aave). The trader doesn't run the swap directly — it submits the swap to its own KeeperHub workflow and gets execution guarantees, gas optimization, and an audit trail. This makes KeeperHub the **mandatory bottom layer** of the BANKON agent economy.

Architecture diff against the existing BANKON spec

Two new components plug in cleanly:

- **`BankonKeeperHubBridge.ts`** is a Node service that exposes three internal endpoints called by the BANKON gateway:
  - `POST /workflow/register-renewal` — creates a KeeperHub workflow when a new subname is minted with auto-renew
  - `POST /workflow/buyback-trigger` — called by the `BankonPaymentRouter` when revenue splits cross thresholds
  - `GET /workflow/:label` — returns the KeeperHub workflow ID associated with a subname

  Uses the KeeperHub MCP HTTP transport with `Authorization: Bearer $KEEPERHUB_API_KEY`.
- **`IBankonKeeperResolver`** is a Solidity interface added to the registrar that lets the resolver write `text("keeperhub.workflow")` and `text("keeperhub.org")` automatically when the agent metadata includes a workflow ID.

The resolver-record schema becomes nine records per subname: `addr`, `text("url")` (mindX endpoint), `text("x402.endpoint")`, `text("algoid.did")`, `text("agent.card")` (ERC-8004 agent-card URI), `text("keeperhub.workflow")`, `text("keeperhub.org")`, `contenthash`,

`addr(node, 0x8000011B)` (Algorand). The `_writeAgentRecords` multicall in `BankonSubnameRegistrar.sol` already includes the new keys (see Part 3).

Code: KeeperHub MCP client and BANKON gateway hook

The TypeScript gateway adds a thin client and a hook in the `/v1/register` flow:

```typescript
```

```typescript
// src/keeperhub.ts
import fetch from "node-fetch";

const KH_BASE = process.env.KEEPERHUB_API_URL ?? "https://app.keeperhub.com";
const KH_KEY  = process.env.KEEPERHUB_API_KEY!;

async function khFetch(path: string, init: any = {}) {
  const r = await fetch(`${KH_BASE}${path}`, {
    ...init,
    headers: {
      ...(init.headers ?? {}),
      Authorization: `Bearer ${KH_KEY}`,
      "Content-Type": "application/json"
    }
  });
  if (!r.ok) throw new Error(`KeeperHub ${path}: ${r.status} ${await r.text()}`);
  return r.json();
}

export async function createRenewalWorkflow(args: {
  label: string;
  ownerAddress: string;
  expiryUnix: number;
  network: string;
}) {
  return khFetch("/api/workflows", {
    method: "POST",
    body: JSON.stringify({
      name: `bankon-renew-${args.label}`,
      description: `Auto-renew ${args.label}.bankon.eth before expiry`,
      nodes: [
        { id: "1", type: "trigger", data: { type: "cron", schedule: "0 0 * * *" } },
        { id: "2", type: "condition", data: {
            check: "compare",
            left:  { kind: "const", value: args.expiryUnix },
            op:    "lte",
            right: { kind: "expr", value: "now() + 30d" }
        } },
        { id: "3", type: "http", data: {
            method: "POST",
            url: `https://registrar.bankon.eth/v1/renew`,
            body: { label: args.label, owner: args.ownerAddress, years: 1 }
        } }
```

```
      ],
      edges: [{from:"1",to:"2"},{from:"2",to:"3"}]
    })
  });
}

export async function executeBuybackSwap(args: {
  network: string;
  usdcAmount: string;
  minPythaiOut: string;
}) {
  return khFetch("/api/direct-execution/contract-call", {
    method: "POST",
    body: JSON.stringify({
      network: args.network,
      contract_address: process.env.COWSWAP_SETTLEMENT!,
      function_name: "setPreSignature",
      function_args: [/* ...orderUid, true */],
    })
  });
}

export async function executeReputationSlash(args: {
  agentNode: string;
  bondAmount: string;
  registrarAddress: string;
  network: string;
}) {
  return khFetch("/api/direct-execution/check-and-execute", {
    method: "POST",
    body: JSON.stringify({
      network: args.network,
      contract_address: process.env.BONAFIDE_REGISTRY!,
      function_name: "isCensured",
      function_args: [args.agentNode],
      condition: { operator: "eq", value: true },
      action: {
        contract_address: args.registrarAddress,
        function_name: "slashBond",
        function_args: [args.agentNode, args.bondAmount]
      }
    })
```

```
  });
}
```

The registrar hook fires after a successful mint:

```typescript
// In gateway /v1/register handler, after L1 tx confirms:
if (req.body.autoRenew) {
  const wf = await createRenewalWorkflow({
    label,
    ownerAddress: owner,
    expiryUnix: expiry,
    network: "sepolia"
  });
  // Patch resolver to include keeperhub.workflow text record
  await registrarContract.setText(node, "keeperhub.workflow", wf.id);
  await registrarContract.setText(node, "keeperhub.org", process.env.KEEPERHUB_ORG_I
}
```

For the *agent calls another agent* beat, the calling agent uses the KeeperHub MCP server directly — Claude Code or any MCP runtime — and a single tool call walks the graph: resolve ENS → read `keeperhub.workflow` → `execute_workflow(wfId, input)`. This is exactly the pattern the KeeperHub README markets as the canonical agent integration.

MCP client configuration

For local use with Claude Code or any MCP-compatible client, add KeeperHub to the MCP server config:

```json
{
  "mcpServers": {
    "keeperhub": {
      "command": "docker",
      "args": ["run", "-i", "--rm", "-e", "KEEPERHUB_API_KEY", "keeperhub-mcp"]
    }
  }
}
```

For HTTP/SSE mode (remote agents):

```bash
docker run -p 3000:3000 \
  -e PORT=3000 \
  -e MCP_API_KEY=your_secure_mcp_key \
  -e KEEPERHUB_API_KEY=your_keeperhub_key \
  keeperhub-mcp
```

All HTTP requests must include `Authorization: Bearer <MCP_API_KEY>`.

Strategic posture for the submission

Submit BANKON to **three** Partner Prizes: ENS (both sub-tracks counted as one selection), KeeperHub, and 0G. The KeeperHub submission framing should be **Focus Area 2: integration** with the bridge to x402 payments, not Focus Area 1, because the brief explicitly weights x402 integration in Area 2 and BANKON's whole architecture is x402-native.

Also submit a `FEEDBACK.md` for the $250 Builder Feedback Bounty. Useful feedback areas include:

- MCP HTTP-mode authentication ergonomics with the `MCP_API_KEY` + `KEEPERHUB_API_KEY` dual-key pattern
- Gaps in workflow-trigger documentation
- Discoverability of the direct-execution endpoints versus workflow endpoints
- Whether `ai_generate_workflow` accepts ENS-aware schemas

What the README needs to say

The README's KeeperHub section must satisfy four checkboxes the judges will literally tick:

1. Working demo link with the KeeperHub workflow visible in the dashboard during the demo
2. The GitHub repo includes the `BankonKeeperHubBridge.ts` and the keeper-hooked Solidity interfaces with tests
3. A `FEEDBACK.md` with at least three specific actionable items (UX, bug, or docs gap)
4. A one-paragraph write-up that says *"BANKON uses KeeperHub as its onchain execution layer for autonomous renewal, PYTHAI buyback-and-make, BONAFIDE-driven slashing, and agent-to-agent x402-paid workflow invocation. Every BANKON subname publishes its KeeperHub workflow ID as an ENS text record, making KeeperHub workflows natively discoverable through ENS resolution."*

Why this is the right second bounty

KeeperHub lets the BANKON demo cross from *"agent has an ENS name"* to *"agent has guaranteed onchain execution rights anchored to that name."* That is the actual product story sponsors care about, and it lifts the ENS submission's quality at the same time — the resolver records become more interesting, the tokenomics become more credible, and the agent-to-agent demo beat becomes physically demonstrable instead of conceptual.

Recommended path forward — prioritized build order

Day-by-day, the hackathon timeline (Apr 24 – May 6, ~12 days) breaks into five phases.

Days 1–2 — Foundation. Register and wrap `bankon-test.eth` on Sepolia, scaffold the Foundry project, get the mainnet-fork test passing with a hardcoded register call against the real NameWrapper. This proves the fuse math works end-to-end and is the highest-risk technical milestone.

Days 3–5 — Core contracts. Implement the four contracts (`Registrar`, `PriceOracle`, `PaymentRouter`, `ReputationGate`) with full happy-path tests and 60%+ branch coverage; deploy to Sepolia; verify on Etherscan.

Days 6–7 — x402 gateway and KeeperHub bridge. Build the gateway in TypeScript (Express + `@coinbase/x402` middleware) wired to the CDP facilitator on Base Sepolia; implement EIP-712 voucher signing; build `BankonKeeperHubBridge.ts` with the three internal endpoints; integrate with mindX `POST /agent/spawn` so spawning an agent triggers the registration flow and creates the renewal workflow.

Days 8–9 — Identity and reputation. Bundle the ERC-8004 Identity mint (deploy a local copy on Sepolia if mainnet ERC-8004 isn't there yet), wire BONAFIDE reputation read/write, and set up the live A2A x402 demo (two mindX agents, one calls the other through KeeperHub).

Days 10–11 — Polish and submission prep. Record the 4-minute demo video following the script in §4; write the README citing the ENS Labs November 2025 ERC-8004 blog and ENSv2 hierarchical-registry rationale; produce the architecture diagram; write `FEEDBACK.md` (KeeperHub requirement); document AI-tool usage; deploy `PYTHAIBuybackKeeper` and `BankonReputationKeeper` workflows on KeeperHub.

Day 12 — Submit. To ENS (Track A and Track B), KeeperHub (Focus Area 2), Builder Feedback Bounty, and 0G; verify all functional-demo links work; commit history shows progressive net-new development; live Sepolia tx hash is on the README front page.

What should NOT slip past v1

Three things should not slip past the v1 hackathon scope:

1. **CCIP-Read offchain resolver** is a v2 — it competes for engineering time with the demo polish that actually wins ENS tracks.
2. **vePYTHAI gauge governance** is post-hackathon — the registrar exposes the `setLengthPrices` / `setSplits` admin surface, but governing it via locked-token voting is a follow-on.
3. **Harberger opt-in mode** for tradable premium names is tantalizing for demo theater but operationally complex; ship the soulbound default in v1 and open the Harberger lane post-hackathon.

The strategic frame

The single most important strategic frame is this: **ENS Labs has published, in print, that the future of agent identity is ENS + ERC-8004 + x402. KeeperHub is publicly building the execution layer for that exact stack. BANKON is the reference implementation that ties them together.** The submission's job is to make the judges feel they are reading their own roadmap.